

Project 2: Patient Triage Conversational Agent (LangGraph)

This project involves creating a stateful conversational agent that can handle non-linear interactions and make decisions.

- **Objective:** To build a stateful, cyclical agent that can triage a patient's reported symptoms, decide on a course of action, and retrieve information from a trusted knowledge base.
- **Core Framework: LangGraph**
- **Real-World Scenario:** A healthcare provider wants to deploy a chatbot on their patient portal. The bot should handle initial patient queries, gathering symptom information and determining if the case requires immediate escalation to a nurse, scheduling a routine check-up, or providing self-care information from approved health guides.
- **Key Tasks & Steps:**
 1. **Graph Design:**
 - Map out the conversation as a state machine. Nodes could include `greet_patient`, `collect_symptoms`, `assess_urgency`, `provide_information`, `schedule_appointment`, and `escalate_to_human`.
 2. **Node Implementation:**
 - Implement each node as a Python function.
 - The `assess_urgency` node will use an LLM call to classify the severity of the collected symptoms.
 - The `provide_information` node will perform RAG on a provided PDF of the clinic's health guides.
 3. **Conditional Edges:**
 - Use LangGraph's conditional edges to create the logic. For example, if `assess_urgency` returns "HIGH", the graph transitions to the `escalate_to_human` node. If it returns "LOW", it transitions to `provide_information`.
 4. **Tool Integration:**
 - Build a vector store from the provided health guide PDFs to serve as the agent's knowledge base.
- **Tech Stack:**
 - **Agent Framework:** LangGraph
 - **Tools:** Python, LangChain (for RAG and chat models), Vector Database (e.g., ChromaDB).

- **LLM:** Any major API-based model.
- **Evaluation: Langfuse** to visualize the path taken through the graph for each conversation. Track metrics like **Context Adherence** and **Faithfulness** for the RAG responses.
- **Demo:**
 - Streamlit UI mimicking the portal
 - Demonstrate different user queries and routing, escalation, advise etc. based on the query
 - Demonstrate email gets sent to the right person (nurse/doctor/ICU/ambulance/reception for appointment scheduling etc. for action) – you can create dummy gmail accounts to demonstrate the same
- **References**

Sending email from an agent- langgraph:
<https://www.youtube.com/watch?v=prbVTy-Y2rQ&t=1s>
https://github.com/tayaln/email-assistant-ai-agent/blob/main/Email_Assistant_AI_Agent.ipynb